Update and Remove Many-to-Many Relationships

Now we'll see how to update and remove these **many-to-many** relationships.

We'll continue from where we left off with the previous code.

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="teams", link_model=HeroTeamLink)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    teams: list[Team] = Relationship(back_populates="heroes", link_model=HeroTeamLink)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
            teams=[team_z_force, team_preventers],
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            teams=[team_preventers],
        )
        hero_spider_boy = Hero(
            name="Spider-Boy", secret_name="Pedro Parqueador", teams=[team_preventers]
        )
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Deadpond:", hero_deadpond)
        print("Deadpond teams:", hero_deadpond.teams)
        print("Rusty-Man:", hero_rusty_man)
        print("Rusty-Man Teams:", hero_rusty_man.teams)
        print("Spider-Boy:", hero_spider_boy)
        print("Spider-Boy Teams:", hero_spider_boy.teams)
```

```python
def main():
    create_db_and_tables()
    create_heroes()


if __name__ == "__main__":
    main()
```

Get Data to Update

Let's now create a function `update_heroes()`.

We'll get **Spider-Boy** and the **Z-Force** team.

As you already know how these goes, I'll use the **short version** and get the data in a single Python statement.

And because we are now using `select()`, we also have to import it.

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select

# Code here omitted 👈

def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()
        team_z_force = session.exec(select(Team).where(Team.name == "Z-Force")).one()

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="teams", link_model=HeroTeamLink)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    teams: list[Team] = Relationship(back_populates="heroes", link_model=HeroTeamLink)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
            teams=[team_z_force, team_preventers],
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            teams=[team_preventers],
        )
        hero_spider_boy = Hero(
            name="Spider-Boy", secret_name="Pedro Parqueador", teams=[team_preventers]
        )
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Deadpond:", hero_deadpond)
        print("Deadpond teams:", hero_deadpond.teams)
        print("Rusty-Man:", hero_rusty_man)
        print("Rusty-Man Teams:", hero_rusty_man.teams)
        print("Spider-Boy:", hero_spider_boy)
        print("Spider-Boy Teams:", hero_spider_boy.teams)
```

```python
def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()
        team_z_force = session.exec(select(Team).where(Team.name == "Z-Force")).one()

        team_z_force.heroes.append(hero_spider_boy)
        session.add(team_z_force)
        session.commit()

        print("Updated Spider-Boy's Teams:", hero_spider_boy.teams)
        print("Z-Force heroes:", team_z_force.heroes)

        hero_spider_boy.teams.remove(team_z_force)
        session.add(team_z_force)
        session.commit()

        print("Reverted Z-Force's heroes:", team_z_force.heroes)
        print("Reverted Spider-Boy's teams:", hero_spider_boy.teams)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

And of course, we have to add `update_heroes()` to our `main()` function:

Python 3.10+

```python
# Code above omitted 👆

def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="teams", link_model=HeroTeamLink)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    teams: list[Team] = Relationship(back_populates="heroes", link_model=HeroTeamLink)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
            teams=[team_z_force, team_preventers],
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            teams=[team_preventers],
        )
        hero_spider_boy = Hero(
            name="Spider-Boy", secret_name="Pedro Parqueador", teams=[team_preventers]
        )
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Deadpond:", hero_deadpond)
        print("Deadpond teams:", hero_deadpond.teams)
        print("Rusty-Man:", hero_rusty_man)
        print("Rusty-Man Teams:", hero_rusty_man.teams)
        print("Spider-Boy:", hero_spider_boy)
        print("Spider-Boy Teams:", hero_spider_boy.teams)
```

```python
def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()
        team_z_force = session.exec(select(Team).where(Team.name == "Z-Force")).one()

        team_z_force.heroes.append(hero_spider_boy)
        session.add(team_z_force)
        session.commit()

        print("Updated Spider-Boy's Teams:", hero_spider_boy.teams)
        print("Z-Force heroes:", team_z_force.heroes)

        hero_spider_boy.teams.remove(team_z_force)
        session.add(team_z_force)
        session.commit()

        print("Reverted Z-Force's heroes:", team_z_force.heroes)
        print("Reverted Spider-Boy's teams:", hero_spider_boy.teams)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

Add Many-to-Many Relationships

Now let's imagine that **Spider-Boy** thinks that the **Z-Force** team is super cool and decides to go there and join them.

We can use the same **relationship attributes** to include `hero_spider_boy` in the `team_z_force.heroes`.

Python 3.10+

```python
# Code above omitted 👆

def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()
        team_z_force = session.exec(select(Team).where(Team.name == "Z-Force")).one()

        team_z_force.heroes.append(hero_spider_boy)
        session.add(team_z_force)
        session.commit()

        print("Updated Spider-Boy's Teams:", hero_spider_boy.teams)
        print("Z-Force heroes:", team_z_force.heroes)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="teams", link_model=HeroTeamLink)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    teams: list[Team] = Relationship(back_populates="heroes", link_model=HeroTeamLink)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
            teams=[team_z_force, team_preventers],
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            teams=[team_preventers],
        )
        hero_spider_boy = Hero(
            name="Spider-Boy", secret_name="Pedro Parqueador", teams=[team_preventers]
        )
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Deadpond:", hero_deadpond)
        print("Deadpond teams:", hero_deadpond.teams)
        print("Rusty-Man:", hero_rusty_man)
        print("Rusty-Man Teams:", hero_rusty_man.teams)
        print("Spider-Boy:", hero_spider_boy)
        print("Spider-Boy Teams:", hero_spider_boy.teams)
```

```python
def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()
        team_z_force = session.exec(select(Team).where(Team.name == "Z-Force")).one()

        team_z_force.heroes.append(hero_spider_boy)
        session.add(team_z_force)
        session.commit()

        print("Updated Spider-Boy's Teams:", hero_spider_boy.teams)
        print("Z-Force heroes:", team_z_force.heroes)

        hero_spider_boy.teams.remove(team_z_force)
        session.add(team_z_force)
        session.commit()

        print("Reverted Z-Force's heroes:", team_z_force.heroes)
        print("Reverted Spider-Boy's teams:", hero_spider_boy.teams)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

Tip

Because we are accessing an attribute in the models right after we commit, with `hero_spider_boy.teams` and `team_z_force.heroes`, the data is refreshed automatically.

So we don't have to call `session.refresh()`.

We then commit the change, refresh, and print the updated **Spider-Boy**'s heroes to confirm.

Notice that we only `add` **Z-Force** to the session, then we commit.

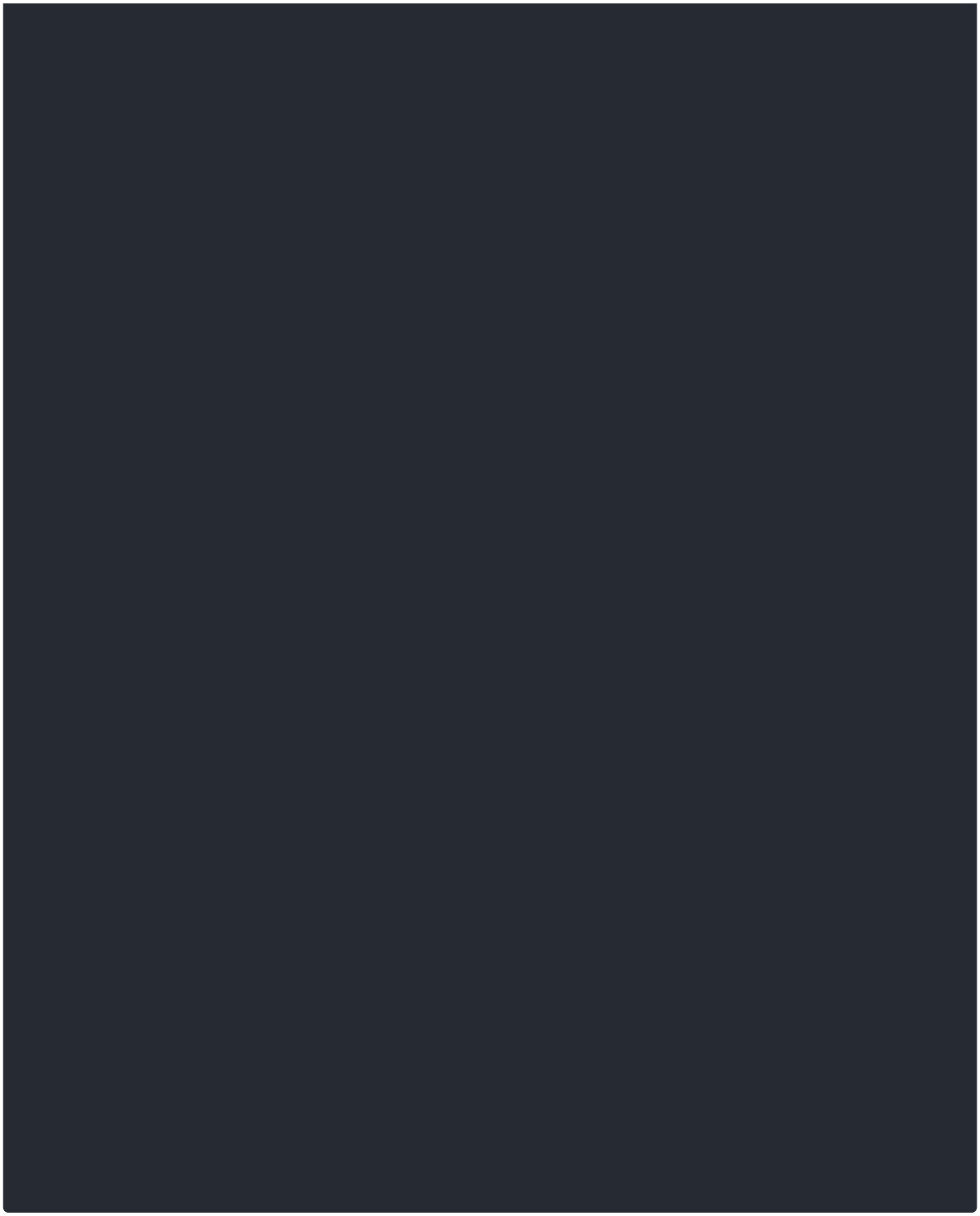
We never add **Spider-Boy** to the session, and we never even refresh it. But we still print his teams.

This still works correctly because we are using `back_populates` in the `Relationship()` in the models. That way, **SQLModel** (actually SQLAlchemy) can keep track of the changes and updates, and make sure they also happen on the relationships in the other related models. 🎉

Run the Program

You can confirm it's all working by running the program in the command line:

```bash
```



Remove Many-to-Many Relationships

Now let's say that right after joining the team, **Spider-Boy** realized that their "life preserving policies" are much more relaxed than what he's used to. 💀

And their *occupational safety and health* is also not as great... 💥

So, **Spider-Boy** decides to leave **Z-Force**.

Let's update the relationships to remove `team_z_force` from `hero_spider_boy.teams`.

Because `hero_spider_boy.teams` is just a list (a special list managed by SQLAlchemy, but a list), we can use the standard list methods.

In this case, we use the method `.remove()`, that takes an item and removes it from the list.

Python 3.10+

```python
# Code above omitted 👆

def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()
        team_z_force = session.exec(select(Team).where(Team.name == "Z-Force")).one()

        team_z_force.heroes.append(hero_spider_boy)
        session.add(team_z_force)
        session.commit()

        print("Updated Spider-Boy's Teams:", hero_spider_boy.teams)
        print("Z-Force heroes:", team_z_force.heroes)

        hero_spider_boy.teams.remove(team_z_force)
        session.add(team_z_force)
        session.commit()

        print("Reverted Z-Force's heroes:", team_z_force.heroes)
        print("Reverted Spider-Boy's teams:", hero_spider_boy.teams)

# Code below omitted 👇
```

👀 Full file preview

Python 3.10+

```python
from sqlmodel import Field, Relationship, Session, SQLModel, create_engine, select


class HeroTeamLink(SQLModel, table=True):
    team_id: int | None = Field(default=None, foreign_key="team.id", primary_key=True)
    hero_id: int | None = Field(default=None, foreign_key="hero.id", primary_key=True)


class Team(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    headquarters: str

    heroes: list["Hero"] = Relationship(back_populates="teams", link_model=HeroTeamLink)


class Hero(SQLModel, table=True):
    id: int | None = Field(default=None, primary_key=True)
    name: str = Field(index=True)
    secret_name: str
    age: int | None = Field(default=None, index=True)

    teams: list[Team] = Relationship(back_populates="heroes", link_model=HeroTeamLink)


sqlite_file_name = "database.db"
sqlite_url = f"sqlite:///{sqlite_file_name}"

engine = create_engine(sqlite_url, echo=True)


def create_db_and_tables():
    SQLModel.metadata.create_all(engine)


def create_heroes():
    with Session(engine) as session:
        team_preventers = Team(name="Preventers", headquarters="Sharp Tower")
        team_z_force = Team(name="Z-Force", headquarters="Sister Margaret's Bar")

        hero_deadpond = Hero(
            name="Deadpond",
            secret_name="Dive Wilson",
            teams=[team_z_force, team_preventers],
        )
        hero_rusty_man = Hero(
            name="Rusty-Man",
            secret_name="Tommy Sharp",
            age=48,
            teams=[team_preventers],
        )
        hero_spider_boy = Hero(
            name="Spider-Boy", secret_name="Pedro Parqueador", teams=[team_preventers]
        )
        session.add(hero_deadpond)
        session.add(hero_rusty_man)
        session.add(hero_spider_boy)
        session.commit()

        session.refresh(hero_deadpond)
        session.refresh(hero_rusty_man)
        session.refresh(hero_spider_boy)

        print("Deadpond:", hero_deadpond)
        print("Deadpond teams:", hero_deadpond.teams)
        print("Rusty-Man:", hero_rusty_man)
        print("Rusty-Man Teams:", hero_rusty_man.teams)
        print("Spider-Boy:", hero_spider_boy)
        print("Spider-Boy Teams:", hero_spider_boy.teams)
```

```python
def update_heroes():
    with Session(engine) as session:
        hero_spider_boy = session.exec(
            select(Hero).where(Hero.name == "Spider-Boy")
        ).one()
        team_z_force = session.exec(select(Team).where(Team.name == "Z-Force")).one()

        team_z_force.heroes.append(hero_spider_boy)
        session.add(team_z_force)
        session.commit()

        print("Updated Spider-Boy's Teams:", hero_spider_boy.teams)
        print("Z-Force heroes:", team_z_force.heroes)

        hero_spider_boy.teams.remove(team_z_force)
        session.add(team_z_force)
        session.commit()

        print("Reverted Z-Force's heroes:", team_z_force.heroes)
        print("Reverted Spider-Boy's teams:", hero_spider_boy.teams)


def main():
    create_db_and_tables()
    create_heroes()
    update_heroes()


if __name__ == "__main__":
    main()
```

And this time, just to show again that by using `back_populates` **SQLModel** (actually SQLAlchemy) takes care of connecting the models by their relationships, even though we performed the operation from the `hero_spider_boy` object (modifying `hero_spider_boy.teams`), we are adding `team_z_force` to the **session**. And we commit that, without even add `hero_spider_boy`.

This still works because by updating the teams in `hero_spider_boy`, because they are synchronized with `back_populates`, the changes are also reflected in `team_z_force`, so it also has changes to be saved in the DB (that **Spider-Boy** was removed).

And then we add the team, and commit the changes, which updates the `team_z_force` object, and because it changed the table that also had a connection with the `hero_spider_boy`, it is also marked internally as updated, so it all works.

And then we just print them again to confirm that everything worked correctly.

Run the Program Again

To confirm that this last part worked, you can run the program again, it will output something like:

```bash
```

Recap

Updating and removing many-to-many relationships is quite straightforward after setting up the **link model** and the relationship attributes.

You can just use common list operation. 🚀